

Introduction à l'instrumentation dynamique binaire avec QBDI

Victor Houal
Daniel Janson

2026-06-27
LeHack, Paris

Quarkslab

L'instrumentation dynamique binaire

Principe d'une DBI

Qu'est-ce qu'une DBI ?



Définition

L'**instrumentation dynamique binaire** (DBI) consiste à injecter du code d'analyse directement dans un binaire (ou une bibliothèque) **pendant son exécution**, sans disposer du code source ni recompiler le programme.

- Le binaire cible continue de s'exécuter normalement : la DBI s'intercale à certains points pour observer ou modifier son comportement
- S'oppose à l'instrumentation **statique**, qui réécrit le binaire sur disque avant exécution (ex: ajout de code à la compilation, binary patching)

Pourquoi utiliser une DBI ?

- **Tracer l'exécution** : reconstruire le flot de contrôle, profiler les performances, étudier le comportement d'un programme
- **Suivre les données** : data flow tracking, taint analysis, vérifier les accès mémoire
- **Contourner des protections** : désobfuscation, unpacking, neutraliser des vérifications anti-debug ou des protections logicielles

Deux familles de techniques

JIT-DBT

Recompilation à la volée des blocs de base dans un *code cache*, instrumentation intercalée.

- Granularité fine
- Surcoût notable

Exemple : Pin, DynamoRIO, Valgrind, QBDI

Hooking inline

Détournement du prologue d'une fonction vers un *trampoline*.

- Léger et ciblé
- Granularité à la fonction

Exemple : *Interceptor* de Frida

Panorama des frameworks de DBI

Framework	Licence	Architectures	OS / mobile	Technique	API / scripting
Intel Pin	Propriétaire	x86 / x86-64	Windows, Linux, macOS	JIT-DBT	C/C++ (<i>Pintools</i>)
DynamoRIO	BSD-3	x86/x86-64, ARM,	Windows, Linux, Android	JIT-DBT	C/C++ (<i>clients</i>)
Valgrind	GPLv2	x86/x86-64, ARM,	Linux, Android	JIT via IR VEX	C (greffons compilés)
Frida	wxWindows (≈ LGPL)	x86/x86-64, ARM, AArch64	Win, Linux, macOS, Android, iOS	Hooking inline + recompilation	JS/TypeScript, Python, C, Swift...
QBDI	Apache 2.0	x86/x86-64, ARM, AArch64	Linux, macOS, Win, Android, iOS	JIT-DBT	C/C++, Python, JS/ Frida

Table 1: Principaux frameworks de DBI.

Nuances sur le tableau

- **Pin** : portage ARM (2006) abandonné, x86 uniquement, propriétaire malgré la gratuité
- **Valgrind** : fort surcoût (IR lourd), pas vraiment scriptable (outils en C compilés, type Memcheck)
- **Frida** : davantage un toolkit de hooking que de la DBT pure, taillé pour le mobile et le reverse applicatif
- **QBDI** : pas d'injecteur intégré (QBDIPreload ou Frida requis), pas de gestion native des signaux/threads/exceptions C++, support iOS expérimental^(exp.)

Limites de la DBI (en général)

- **Surcoût** : chaque instruction instrumentée passe par du code additionnel, l'exécution peut être ralentie d'un facteur 10x à 100x
- **Détection** : la présence de la DBI dans le même process est détectable (timing checks, vérification de mappings mémoire, self-checksumming)
- **Spécificités d'architecture** : gestion des syscalls, des threads, des exceptions et des signaux qui diffère selon l'OS et l'ISA cible



À noter

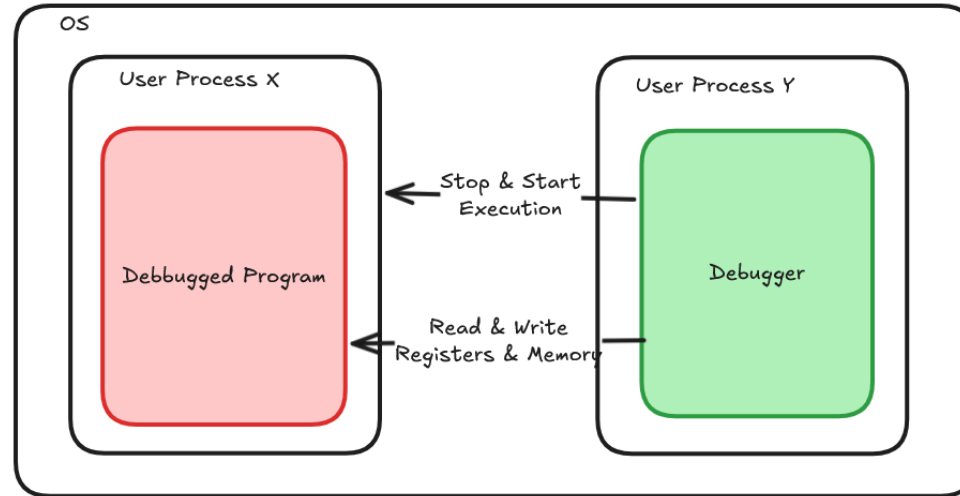
Ces limites sont celles de la DBI comme approche générale.



Fonctionnement d'un débogueur

- Repose sur l'interface de debug du système (ptrace, Win32 Debug API...)
- Isolation mémoire forte : le débogueur et le programme débogué sont deux processus séparés
- Chaque arrêt (breakpoint, single-step) implique un changement de contexte coûteux entre les deux processus

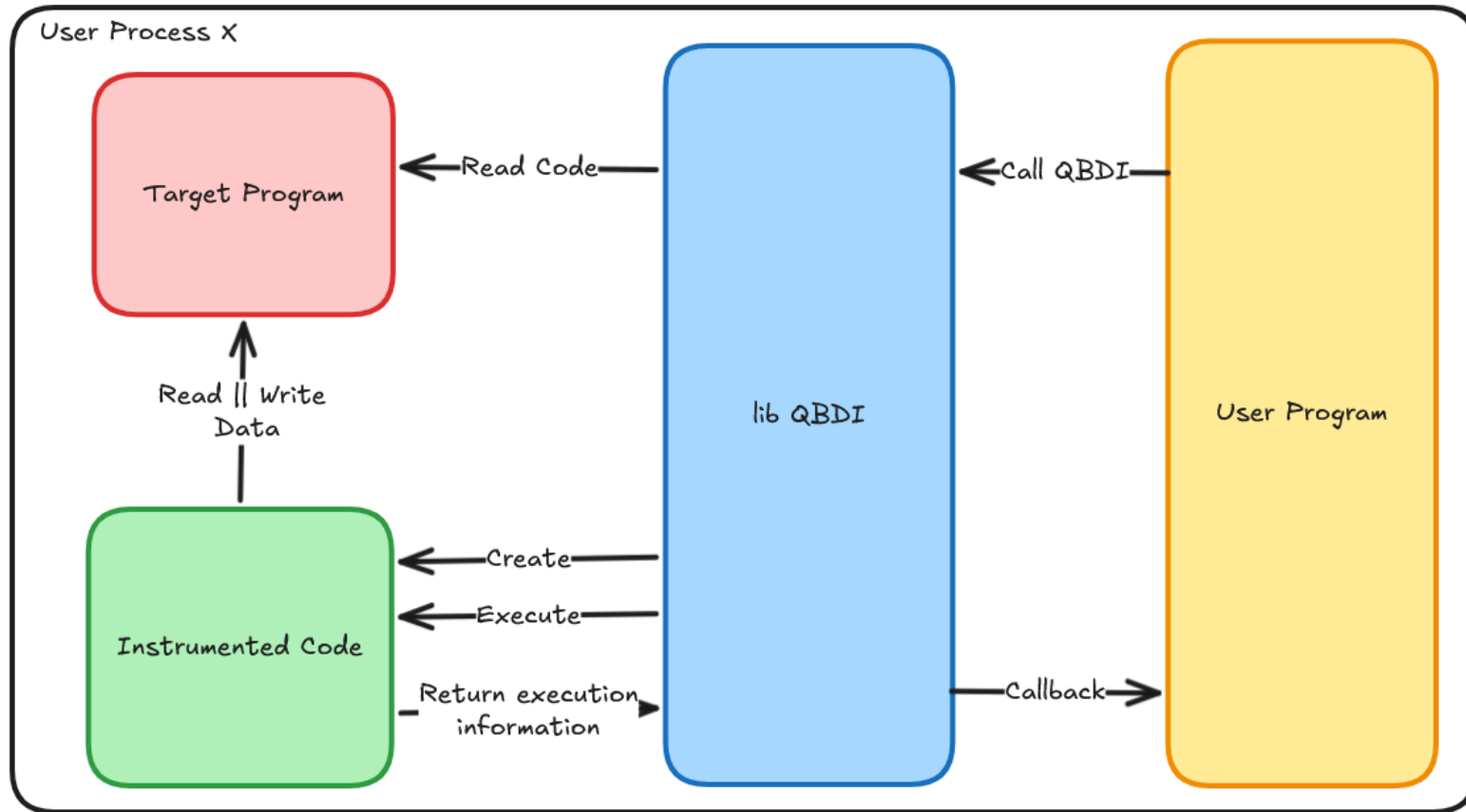
Fonctionnement d'un débogueur



Fonctionnement de QBDI

- QBDI s'exécute **dans le même processus** que la cible, avec le même mapping mémoire
- Beaucoup moins d'interactions avec l'OS qu'un débogueur classique → instrumentation plus rapide
- Contrepartie : il faut injecter la cible et la DBI dans le même processus

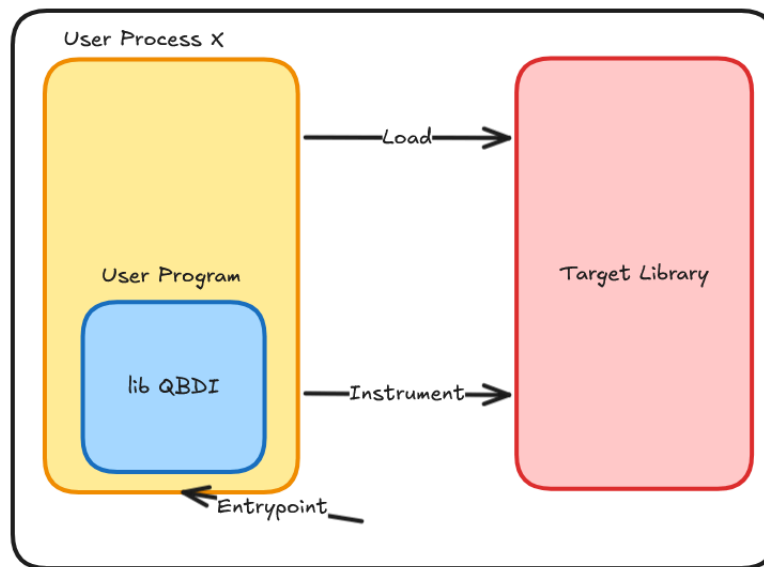
Fonctionnement de QBDI



Injection dans le processus

Importer le code cible dans le process courant

- On charge la bibliothèque cible avec `dlsym/LoadLibrary`
- QBDI instrumente ensuite directement la méthode visée
- Approche la plus simple, mais nécessite de pouvoir charger la cible comme une bibliothèque (pas toujours possible pour un exécutable autonome)

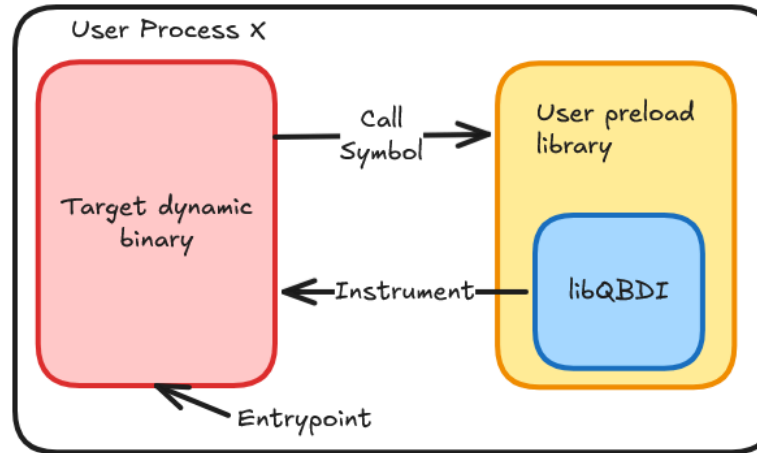




Précharger QBDI dans le programme cible

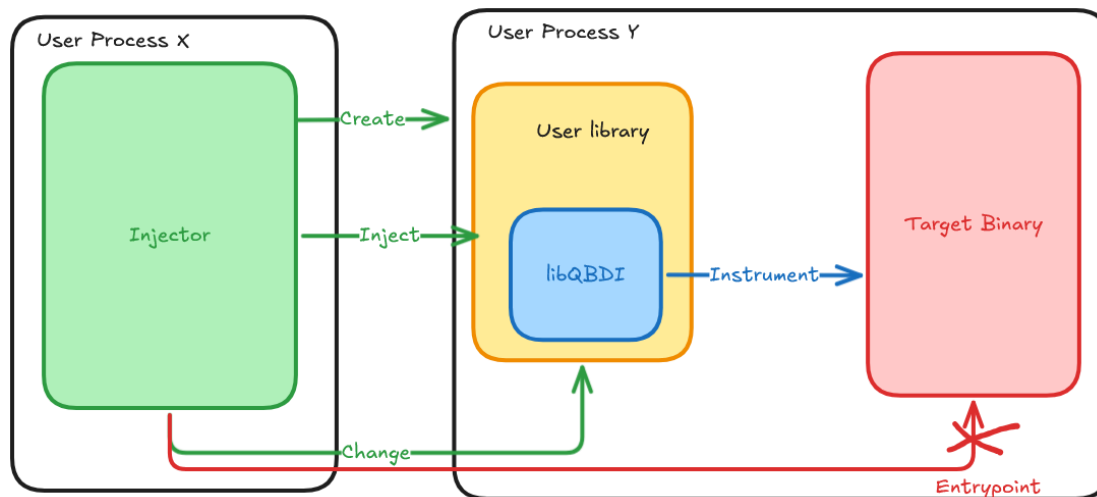
- Le programme cible est lancé avec la bibliothèque QBDI préchargée (`LD_PRELOAD` sous Linux)
- Cette bibliothèque exporte un symbole utilisé par le programme cible, et instrumente le reste des instructions de la cible
- Nécessite que le processus cible utilise la libc, afin de hooker le symbole `__libc_start_main` et de prendre la main avant le code utilisateur

Précharger QBDI dans le programme cible



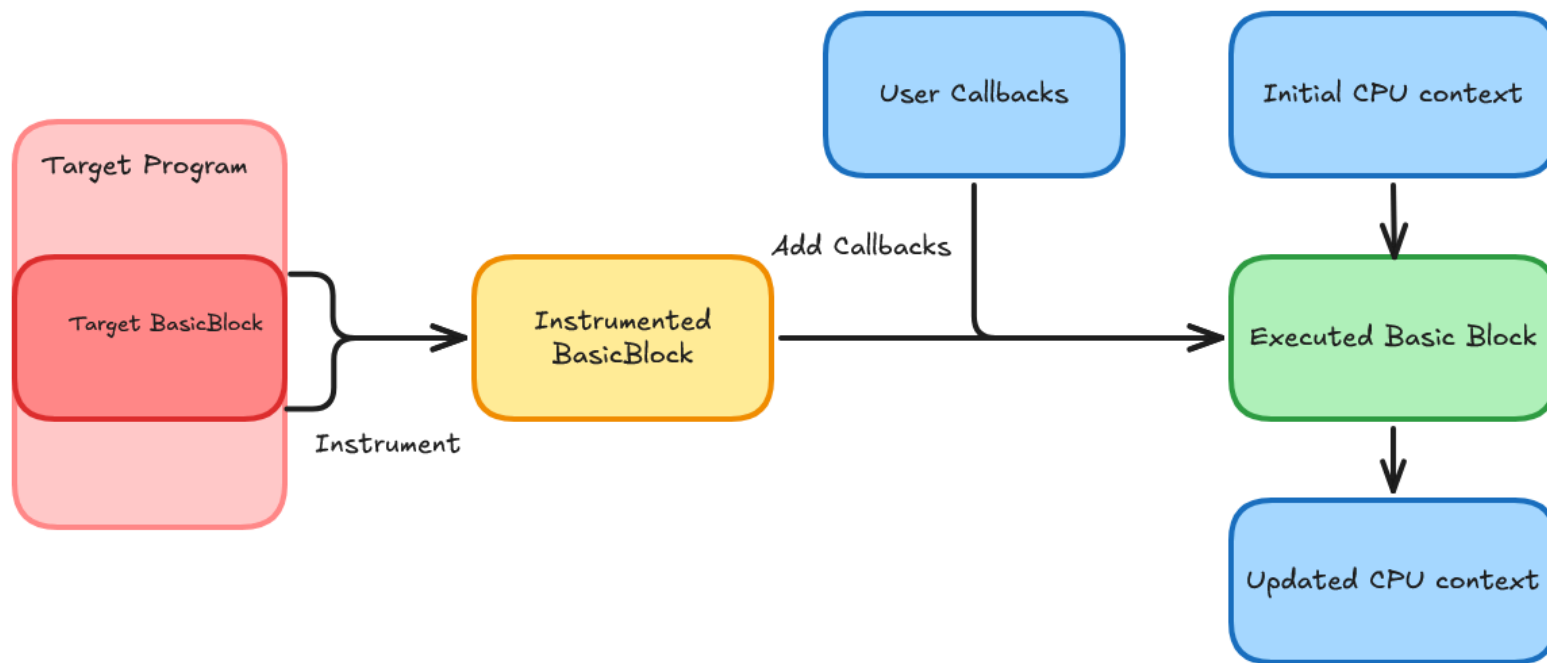
Injecter QBDI avec un injecteur

- Disponible via le binding Frida
- Utilise l'API de débogage du système pour charger QBDI dans le processus cible et rediriger son point d'entrée



Processus d'instrumentation

Instrumentation d'un BasicBlock



Étape interne : relocation des instructions



Avant d'être exécutées depuis leur nouvel emplacement, les instructions sont modifiées par QBDI pour conserver leur comportement d'origine (ex: recalcul des adresses relatives, des accès PC-relatifs).

```
0x360a:  lea    rsi,[rax+0x18]
0x360e:  mov    rdx,QWORD PTR [rbp-0x18]
0x3612:  mov    rax,QWORD PTR [rbp-0x8]
0x3616:  mov    rcx,QWORD PTR [rip+0x319bb]
0x361d:  mov    r8,rcx
0x3620:  mov    rcx,rdx
0x3623:  mov    rdx,rdi
0x3626:  mov    rdi,rax
0x3629:  call   3240
```

```
0x360a:  lea    rsi,[rax+0x18]
0x360e:  mov    rdx,QWORD PTR [rbp-0x18]
0x3612:  mov    rax,QWORD PTR [rbp-0x8]
0x3616:  mov    rcx,QWORD PTR [rip+0x319bb]
0x361d:  mov    r8,rcx
0x3620:  mov    rcx,rdx
0x3623:  mov    rdx,rdi
0x3626:  mov    rdi,rax
0x3629:  call   3240
```

Ajout des callbacks utilisateur

Les callbacks spécifiés par l'utilisateur sont insérés entre chaque instruction du bloc relocalisé, permettant d'observer ou de modifier l'état du programme à granularité fine.

```
0x360a:  lea    rsi,[rax+0x18]
0x360e:  mov    rdx,QWORD PTR [rbp-0x18]
0x3612:  mov    rax,QWORD PTR [rbp-0x8]
0x3616:  mov    rcx,QWORD PTR [rip+0x319bb]
0x361d:  mov    r8,rcx
0x3620:  mov    rcx,rdx
0x3623:  mov    rdx,rdi
0x3626:  mov    rdi,rax
0x3629:  call   3240
```

```
QBDI      Callback
0x360a:  lea    rsi,[rax+0x18]
QBDI      Callback
0x360e:  mov    rdx,QWORD PTR [rbp-0x18]
QBDI      Callback
0x3612:  mov    rax,QWORD PTR [rbp-0x8]
QBDI      Callback
0x3616:  mov    rcx,QWORD PTR [rip+0x319bb]
QBDI      Callback
0x361d:  mov    r8,rcx
QBDI      Callback
0x3620:  mov    rcx,rdx
QBDI      Callback
0x3623:  mov    rdx,rdi
QBDI      Callback
0x3626:  mov    rdi,rax
QBDI      Callback
0x3629:  call   3240
QBDI      Callback
```

Ajout des callbacks mémoire

Les callbacks mémoire ne peuvent être ajoutés que sur les instructions réalisant un accès mémoire (MemoryAccess), ou sur un sous-ensemble d'instructions spécifiques supportées par QBDI.

```
0x360a:  lea    rsi,[rax+0x18]
0x360e:  mov    rdx,QWORD PTR [rbp-0x18]
0x3612:  mov    rax,QWORD PTR [rbp-0x8]
0x3616:  mov    rcx,QWORD PTR [rip+0x319bb]
0x361d:  mov    r8,rcx
0x3620:  mov    rcx,rdx
0x3623:  mov    rdx,rdi
0x3626:  mov    rdi,rax
0x3629:  call   3240
```

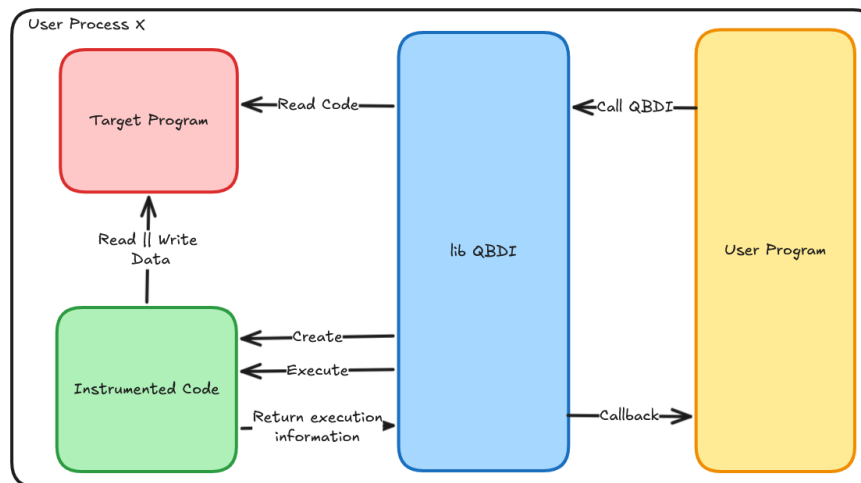
```
0x360a:  lea    rsi,[rax+0x18]
QBDI    Memory Read Callback
0x360e:  mov    rdx,QWORD PTR [rbp-0x18]
QBDI    Memory Read Callback
0x3612:  mov    rax,QWORD PTR [rbp-0x8]
QBDI    Memory Read Callback
0x3616:  mov    rcx,QWORD PTR [rip+0x319bb]
0x361d:  mov    r8,rcx
0x3620:  mov    rcx,rdx
0x3623:  mov    rdx,rdi
0x3626:  mov    rdi,rax
0x3629:  call   3240
QBDI    Memory Write Callback
```

QBDI

Présentation du projet

Informations principales

- Projet open source (licence Apache 2.0), maintenu par Quarkslab
- Écrit majoritairement en C++
- Construit au-dessus de **LLVM** (réutilise son désassembleur et son assembleur pour générer le code instrumenté)
- Supporte les architectures ARM32, AArch64, x86 et x86_64
- Multiplateforme : Windows, Linux, OSX, Android, iOS (instable)
- Dernière version publiée : 0.12.1



Interfaces utilisateur

QBDI expose le même moteur d'instrumentation via plusieurs langages, afin de s'adapter aux usages :

- **API C/C++** : interface native, performances maximales, intégration dans d'autres outils (ex: désobfuscateurs, sandbox)
- **Python 3** (via **PyQBDI**) : prototypage rapide de scripts d'analyse, le binding utilisé pendant ce workshop
- **JS** (via **Frida/QBDI**) : combine l'écosystème de scripting de Frida avec le moteur d'instrumentation de QBDI

QBDI 101

Installer PyQBDI

Pré-requis

- Linux x86_64 (Debian 13 ou Arch Linux recommandés pour le workshop)
- Python 3, `pip`
- Aucune compilation nécessaire : le binding Python embarque une build précompilée de QBDI

Installation



```
1 pip install PyQBDI
```



- Vérifier l'installation :

```
1 python3 -c "import pyqbdi; print(pyqbdi.__version__)"
```



Lancer un script PyQBDI sur un binaire

QBDI doit cohabiter dans le même process que sa cible. Pour un exécutable simple, on utilise **QBDIPreload** :

```
1 python3 -m pyqbdipreload mon_script.py -- ./target arg1 arg2
```



- `mon_script.py` est chargé **avant** `main()` de la cible
- Tout ce qui suit `--` est transmis tel quel à la cible (`argv`)

Point d'entrée d'un script QBDIPreload

```
1 def pyqbdipreload_on_run(vm, start, stop):  
2     ...
```

 Python

- `vm` : instance de `pyqbdipreload.VM`, pilote l'exécution instrumentée
- `start` : adresse de l'entrypoint de la cible
- `stop` : adresse de retour (fin de l'exécution instrumentée)



À retenir

Sans appel à `vm.run(start, stop)`, la cible ne s'exécute jamais : c'est cette ligne qui rend la main à la cible, sous contrôle de la VM.

**Principe : enregistrer des
callbacks**



Le cycle de vie d'un callback

1. On enregistre un callback auprès de la VM (`vm.addXxxCB( . . . )`), en précisant **quand** il doit se déclencher (adresse, plage, type d'évènement)
2. La VM rend la main à la cible via `vm.run(start, stop)`
3. À chaque déclenchement, la VM interrompt l'exécution JIT et appelle la fonction Python correspondante
4. Le callback retourne une **action** à la VM : `pyqbd1.CONTINUE` (poursuivre normalement), ou d'autres valeurs pour modifier le flot

Signature d'un callback

```
1 def cbk(vm, gpr, fpr, data):  
2     ...  
3     return pyqbd1.CONTINUE
```

 Python

- `gpr` / `fpr` : état des registres au moment du déclenchement (entiers / flottants)
- `data` : objet passé librement par l'utilisateur à l'enregistrement, réutilisable pour accumuler un résultat
- C'est la signature commune à la plupart des `vm.addXxxCB(...)` (`addCodeAddrCB`, `addCodeRangeCB`, `addMemRangeCB`...)



À noter

Toutes les familles de callbacks ne partagent pas exactement cette signature : `addVMEventCB`, par exemple, insère un paramètre supplémentaire. On le verra à l'exercice 4.

Panorama des callbacks QBDI (1/2)

- Signature (vm, gpr, fpr, data), callback retourne un `pyqbd.VMAAction` (ex: `pyqbd.CONTINUE`)

API	Déclenchement
<code>addCodeCB</code>	chaque instruction exécutée
<code>addCodeAddrCB</code>	une adresse précise
<code>addCodeRangeCB</code>	chaque instruction d'une plage d'adresses
<code>addMnemonicCB</code>	instruction correspondant à un mnémonique (joker *)
<code>addMemAccessCB</code>	tout accès mémoire correspondant au type donné
<code>addMemAddrCB</code>	accès mémoire à une adresse précise
<code>addMemRangeCB</code>	accès mémoire dans une plage d'adresses



Piège fréquent

`addMemAddrCB` et `addMemRangeCB` sont des callbacks **virtuels** : ils passent par un callback générique déclenché sur **chaque** accès mémoire, qui filtre



Panorama des callbacks QBDI (1/2)

Piège fréquent

ensuite l'adresse. Surcoût de performance notable si on en enregistre beaucoup.

Panorama des callbacks QBDI (2/2)

API	Déclenchement	Signature
addVMEventCB	un évènement VM ¹	(vm, vmState, gpr, fpr, data)
addInstrRule / addInstrRuleRange	analyse fine, insère des règles d'instrumentation custom	(vm, instAnalysis, data)

- addVMEventCB insère vmState (l'évènement) avant gpr/fpr
- addInstrRule est d'une autre nature : le callback ne renvoie pas une action mais une **liste de règles** (InstrRuleDataCBK) à appliquer à l'instruction analysée, usage avancé, hors périmètre d'aujourd'hui

¹ex: BASIC_BLOCK_ENTRY, EXEC_TRANSFER_CALL

Panorama des callbacks QBDI (2/2)



Le point commun

Pour l'essentiel des callbacks (catégorie 1/2), on retrouve toujours `vm` (piloter la VM), `gpr/fpr` (état courant) et `data` (contexte utilisateur). `addVMEventCB` et `addInstrRule*` s'en écartent, chacun à sa façon.

Exercice 1 - Dump des registres



Cible

fib(10), fonction @ 0x00401196

- Observer l'état des registres à chaque appel de fib
- API : `vm.addCodeAddrCB(address, position, callback, data)`

```
1 vm.addCodeAddrCB(0x00401196, pyqbd1.PREINST, cbk, None)
```

 Python



addCodeAddrCB en détail

- `address` : adresse exacte où déclencher le callback
- `position` : `pyqbdì.PREINST` (avant l'instruction) ou `pyqbdì.POSTINST` (après)
- `callback` : fonction (`vm, gpr, fpr, data`)
- `data` : valeur arbitraire transmise au callback à chaque appel
- Les champs de `gpr` sont accessibles comme attributs : `gpr.rdi`, `gpr.rax`, `gpr.rsp`, ...

```
1 def cbk(vm, gpr, fpr, data):  
2     ...  
3     return pyqbdì.CONTINUE
```

 Python



Questions

- Combien de fois `fib` est-elle appelée au total pour `fib(10)` ?
- Quel registre porte l'argument ? Lequel porte la valeur de retour ?

Éditez `ex1_gpr.py`, lancez avec `pyqbdipreload`.

Exercice 2 - Trace d'instructions



Cible

insertion_sort @ 0x004011d1

- Capturer chaque instruction exécutée dans une plage de code
- API : `vm.addCodeRangeCB(start, end, position, callback, data)`
- Nouvelle méthode : `vm.getInstAnalysis()`, analyse de l'instruction courante

```
1 vm.addCodeRangeCB(0x004011d1, 0x00401293, pyqbd1.PREINST, cbk, None)
```

 Python



InstAnalysis, votre désassembleur embarqué

- `vm.getInstAnalysis()` renvoie un objet décrivant l'instruction qui va être exécutée (ou qui vient de l'être, selon `PREINST/POSTINST`)
- Champs utiles : `.address`, `.disassembly`, `.mnemonic`, `.instSize...`

```
1 def cbk(vm, gpr, fpr, data):  
2     inst = vm.getInstAnalysis()  
3     ...  
4     return pyqbd1.CONTINUE
```

 Python



Question

Combien d'instructions sont exécutées en tout ? Repérez-vous la boucle interne dans la trace ?

Exercice 3 - Surveiller les accès mémoire



Cible

`compute_score(Player*)`, struct mappée à l'adresse fixe `0xDEAD0000`

- Observer les lectures mémoire dans une plage d'adresses
- API : `vm.addMemRangeCB(start, end, type, callback, data)`
- Nouvelle méthode : `vm.getInstMemoryAccess()`, liste des accès de l'instruction courante

```
1 vm.addMemRangeCB(START_ADDR, END_ADDR, pyqbdm.MEMORY_READ, cbk, None)
```

 Python

MemoryAccess, anatomie d'un accès

- `type` : combinaison de bits, à tester avec `&`, `pyqbd1.MEMORY_READ`, `pyqbd1.MEMORY_WRITE`
- `accessAddress` : adresse accédée
- `size` : taille de l'accès en octets
- `value` : valeur lue ou écrite

```
1 def cbk(vm, gpr, fpr, data):  
2     for acc in vm.getInstMemoryAccess():  
3         ...  
4     return pyqbd1.CONTINUE
```

 Python

Question

Dans quel ordre les champs de la structure sont-ils lus ? Peut-on en déduire la disposition (offsets, tailles) sans regarder le source ?

Exercice 4 - Couverture par bloc de base



Cible

`analyze_str`, automate à états traitant une chaîne caractère par caractère

- Compter les passages dans chaque **basic block** exécuté
- API : `vm.addVMEventCB(event, callback, data)`, callback d'évènement, pas d'instruction
- `pyqbdi.BASIC_BLOCK_ENTRY` : déclenché à l'entrée de chaque nouveau bloc
- `vmState.basicBlockStart` : adresse du bloc qui vient d'être atteint

```
1 coverage = {}  
2 vm.addVMEventCB(pyqbdi.BASIC_BLOCK_ENTRY, cbk_bb, coverage)
```





Pourquoi un dictionnaire comme data ?

- `data` est passé **par référence** à chaque déclenchement : un objet mutable (dict, liste...) permet d'accumuler un résultat sur toute l'exécution, sans variable globale
- `vmState` (et non `gpr`) en 2e position : signature propre aux callbacks d'évènement VM

```
1 def cbk_bb(vm, vmState, gpr, fpr, data):  
2     addr = vmState.basicBlockStart  
3     ...  
4     return pyqbdm.CONTINUE
```

 Python



Question

Quel basic block est le plus exécuté ? Qu'est-ce que cela révèle sur la structure interne de la fonction ?

Exercice 5 - Modifier l'état du programme



Cible

`check_license("DEAD-BEEF-C0DE")` retourne `0` (INVALID)

- Jusqu'ici, les callbacks ne faisaient qu'**observer**. Ici on modifie l'état du programme depuis le callback.
- Les champs de `gpr` sont accessibles en écriture (ex: `gpr.rax = 1`)

```
1 def cbk(vm, gpr, fpr, data):  
2     gpr.rax = ...  
3     return pyqbd1.CONTINUE
```

Python



Tip

Il est possible d'utiliser `setGRPState(gpr)`, mais ce n'est pas obligatoire.



Questions

- À quel(s) endroit(s) de la fonction peut-on intercepter l'exécution pour inverser le résultat ?
- Existe-t-il plusieurs stratégies valables ?

Récapitulatif des APIs vues

API	Usage
<code>vm.addCodeAddrCB</code>	callback sur une adresse précise
<code>vm.addCodeRangeCB</code>	callback sur toute instruction d'une plage
<code>vm.addMemRangeCB</code>	callback sur accès mémoire dans une plage
<code>vm.addVMEventCB</code>	callback sur évènement VM (ex: <code>BASIC_BLOCK_ENTRY</code>)
<code>vm.getInstAnalysis</code>	analyse de l'instruction courante
<code>vm.getInstMemoryAccess</code>	liste des accès mémoire de l'instruction courante
<code>vm.getGPRState / setGPRState</code>	lire / écrire l'état des registres
<code>vm.run(start, stop)</code>	rend la main à la cible sous contrôle de la VM

Contourner l'anti-désassemblage

**Pourquoi le désassemblage
statique peut mentir**

Le décodage x86 dépend de la position

- x86_64 est un jeu d'instructions à **longueur variable** (1 à 15 octets), sans contrainte d'alignement
- Rien dans le flux d'octets n'indique où commence une instruction : le CPU décode simplement ce qui se trouve à l'adresse pointée par RIP



Consequence

Les mêmes octets peuvent se décoder en instructions différentes selon l'octet de départ. Le flux n'a pas de découpage "correct" en soi, seul RIP en décide à l'exécution.



Deux stratégies de désassemblage statique

Balayage linéaire

`objdump -d`, vue linéaire de Ghidra

- Décode octet après octet, en supposant que chaque instruction suit directement la précédente
- Trivialement trompé par un saut au-dessus d'un octet parasite

Parcours récursif

IDA Pro, analyseur de Ghidra

- Suit le flot de contrôle : ne décode une adresse que si une arête de contrôle de flot y mène
- Résiste au saut inconditionnel simple, mais pas à un **prédicat opaque**

La technique : prédicat opaque + branchement conditionnel

```
1 xor    r11d, r11d    ; prédicat opaque : ZF = 1, toujours vrai
2 jz     real          ; toujours pris à l'exécution, mais CONDITIONNEL
3 .byte  0xE8          ; octet parasite : opcode de CALL rel32 (5 octets)
4 real:
5 <instruction réelle>
```

- Le saut est **conditionnel** : un désassembleur à parcours récursif ne peut pas prouver que le prédicat est toujours vrai, il doit donc décoder aussi le chemin mort
- `0xE8` est `CALL rel32` (5 octets) : décoder à partir de cette adresse avale les octets suivants (la vraie instruction) dans un faux `call`

Ce que voit l'outil statique vs le CPU

Vue statique (menteuse)

```
1 0x401424: 74 01
2          je 0x401427
3 0x401426: e8 88 d1 d2 c0
4          call 0xfffffffffc112e5b3
5 0x40142b: 88 45 cb
6          mov [rbp-0x35], al
```

Un `call` vers une adresse aberrante, qui ne s'exécute jamais.

Vue CPU / QBDI (réelle)

```
1 0x401427: mov ...
2 0x401429: rol al, cl
```

Le prédicat est évalué, le saut est pris, l'octet `0xE8` n'est jamais lu.

Pourquoi QBDI n'est pas dupé

- QBDI est une instrumentation **dynamique** : il décode les basic blocks le long du chemin que le CPU emprunte réellement
- Il évalue `xor r11d, r11d ; jz` pour de vrai, voit `ZF=1`, suit le saut
- L'octet `0xE8` n'est jamais sur un chemin exécuté : QBDI ne le décode jamais, et ne se désaligne donc jamais



À retenir

Les outils statiques décodent **où les octets pourraient être atteignables**.
QBDI décode **où RIP va réellement**. L'obfuscation par instructions chevauchantes vit dans cet écart entre statique et dynamique.

Exercice 2 - Casser un keygen obfusqué



Cible

`./target <clé de 16 octets>`

- Chaque octet de la clé passe par une chaîne de **5 opérations**, le résultat est comparé à une table cible
- Les octets sont **chaînés** : la sortie de l'un nourrit l'entrée du suivant, impossible de bruteforcer indépendamment
- Les fonctions concernées sont cachées derrière la technique précédente



Retrouver les fonctions dans le binaire

- Le binaire est compilé `-no-pie` : les adresses sont fixes

```
1 nm build/target | grep -iE 'transform|check|load'  
2 objdump -d -M intel build/target
```



- Repérer le début et la fin de chaque fonction d'intérêt (`load_k_table`, `load_add_table`, `transform_byte`, `check_key`)



Tracer le flot réel d'exécution

- Même méthode que l'exercice 1 : `addCodeRangeCB` sur la plage couvrant les fonctions ciblées, et `vm.getInstAnalysis()` dans le callback

```
1 vm.addCodeRangeCB(L0, HI, pyqbdι.PREINST, cb, None)
```

 Python

- QBDI résout les sauts du prédicat opaque : la trace affiche les instructions réelles, jamais de faux `call`



Option A - filtrer la chaîne désassemblée

- Dans un callback générique (addCodeRangeCB), tester le mnémonique et la présence d'un registre 8 bits dans `inst.disassembly`

```
1 def cb(vm, gpr, fpr, data):
2     inst = vm.getInstAnalysis()
3     dis = inst.disassembly
4     if ...: # filtrer sur le mnémonique ET la taille d'opérande
5         print(inst.address, dis)
6     return pyqbdm.CONTINUE
```

 Python

- Simple, mais demande de parser une chaîne de caractères à chaque instruction

Option B - filtrer avec addMnemonicCB

- QBDI peut filtrer **lui-même**, par mnémonique LLVM, sans jamais invoquer le callback sur le bruit
- API : `vm.addMnemonicCB(pattern, position, callback, data)`

```
1 vm.addMnemonicCB("ROL8*", pyqbdι.PREINST, cb, None)
```

[Python](#)

- Les mnémonics LLVM encodent déjà la taille d'opérande (ROL8rCL, XOR8rr, ADD8rr, CMP8rr...) : un seul critère suffit, plus besoin de tester la largeur du registre dans la chaîne désassemblée
- Un appel par motif à surveiller (ROL8*, ROR8*, XOR8*, ADD8*, CMP8*, SETCC*...)



À retenir

`addMnemonicCB` déplace le filtrage de l'espace utilisateur vers la VM : le callback n'est même pas appelé pour les instructions qui ne correspondent pas au motif, là où l'option A les laisse passer puis les rejette après coup.



Questions

- Quelles sont les 5 opérations appliquées à chaque octet, dans quel ordre, avec quelles constantes par position ?
- Qu'est-ce qui chaîne un octet au suivant ?
- En inversant cette transformation, quelle est la clé de 16 octets ?

Éditez `ex2_trace.py`, lancez avec `pyqbdipreload`.

Inverser la transformation

- Une fois les 5 étapes et leurs constantes connues, il suffit de les défaire dans l'ordre inverse, en partant de la table cible
- Le chaînage (prev) se résout naturellement de gauche à droite : la valeur précédente est toujours connue à l'étape suivante



Piège fréquent

Une rotation ou un xor s'inverse facilement, mais il faut inverser les étapes dans l'ordre **inverse** de leur application (5 -> 1), pas dans l'ordre d'origine.

Instrumenter une bibliothèque partagée

Une cible sans main

Le problème

- Jusqu'ici, `pyqbdipreload` nous donnait un `start/stop` tout prêt, fournis par le point d'entrée du programme
- Une bibliothèque partagée n'a pas de `main` : on ne peut pas la “lancer”, seulement appeler une de ses fonctions exportées
- Il faut donc construire nous-mêmes le contexte d'appel : une pile, un état de registres, une fausse adresse de retour



Quel mode d'injection ?

Sur un exécutable dynamique avec libc, `pyqbdipreload` hooke `__libc_start_main`. Ici, pas de `main` à intercepter : on revient au premier mode vu en introduction, **charger la cible dans le process courant** (`dlsym/ctypes.CDLL`), QBDI n'instrumente alors que la méthode visée.



Charger la bibliothèque et trouver l'entrée

```
1 import ctypes
2 import pyqbd
3
4 lib = ctypes.CDLL("librasp.so")
5 entry = ctypes.cast(lib.rasp_protect, ctypes.c_void_p).value
```

 Python

- `ctypes.CDLL` charge la bibliothèque dans le process courant
- `ctypes.cast(..., c_void_p).value` récupère l'adresse réelle du symbole

Appeler une fonction sous QBDI

Construire le contexte d'exécution

- Une bibliothèque appelée ainsi n'a pas de pile : il faut en allouer une
- `vm.addInstrumentedModuleFromAddr` restreint l'instrumentation au seul module visé : le reste (libc, libstdc++) tourne nativement, hors VM
- `pyqbdj.simulateCall` pose une fausse adresse de retour, pour que la fonction puisse `ret` vers un endroit connu

```
1 vm = pyqbdj.VM()  
2 state = vm.getGPRState()  
3 stack = pyqbdj.allocateVirtualStack(state, 0x100000)  
4 vm.addInstrumentedModuleFromAddr(entry)  
5 pyqbdj.simulateCall(state, FAKE_RET)  
6 vm.setGPRState(state)  
7 vm.run(entry, FAKE_RET)
```

 Python



Construire le contexte d'exécution



À retenir

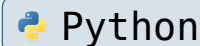
Attacher la VM sans callback ne change rien au résultat de l'appel : on gagne un **poste d'observation**, pas encore une observation.



Une alternative plus directe : `vm.call`

- Pour une fonction avec une signature C classique (arguments scalaires ou pointeurs), `vm.call` évite de poser soi-même les registres d'appel

```
1 success, retval = vm.call(entry, [arg0, arg1, arg2])
```



- Les buffers passés en argument peuvent être alloués de deux manières :
 - **API PyQBDI** — `pyqbdy.allocateMemory()` puis `pyqbdy.writeMemory()` : la voie directe et recommandée
 - **ctypes** — plus contraignant, mais tout à fait faisable ; il faut notamment caster les adresses en `int` avant de les passer à PyQBDI

Exercice 3 - Comprendre un RASP sans le lire



Cible

`librasp.so`, bibliothèque de type RASP (Runtime Application Self Protection), obfusquée et strippée

- Un seul point d'entrée documenté : `rasp_protect()`, qui renvoie un bitmask de menaces détectées
- Lire le binaire statiquement est pénible : flot de contrôle aplati, chaînes reconstruites au runtime, aucun symbole utile
- Un RASP doit **observer le système** : fichiers, environnement, résolution de symboles - l'obfuscation cache la logique, pas les entrées/sorties

Étape 1 - Appel brut



- Appeler `rasp_protect()` directement via `ctypes`, sans instrumentation



Question

Que renvoie l'appel sur cette machine ? Le nombre seul vous dit-il **quelles** vérifications se sont déclenchées, ou **comment** ?

Étape 2 - Capturer les sorties vers la libc



- `pyqbdm.EXEC_TRANSFER_CALL` : évènement VM déclenché à chaque transfert de contrôle **hors** du module instrumenté
- `gpr.rip` contient alors l'adresse réellement appelée

```
1 def on_call(vm, vmState, gpr, fpr, data):  
2     target = gpr.rip  
3     ...  
4     return pyqbdm.CONTINUE  
5  
6 vm.addVMEventCB(pyqbdm.EXEC_TRANSFER_CALL, on_call, None)
```

 Python

- Résoudre l'adresse en nom de symbole avec `dladdr(ctypes.CDLL(None).dladdr)`

Étape 2 - Capturer les sorties vers la libc



Question

Quelles fonctions de la libc sont appelées par `rasp_protect()` ? Les noms seuls suggèrent-ils déjà des catégories de vérifications ?



Étape 3 - Lire les arguments des appels

- Connaître le nom de la fonction appelée ne suffit pas : la vraie information est dans ses **arguments**
- Convention d'appel System V AMD64 : `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
- Lire une chaîne pointée par un registre avec `pyqbd1.readMemory`

```
1 s = pyqbd1.readMemory(gpr.rdi, 64).split(b"\x00")[0]
```

 Python



Piège fréquent

Ne lire comme pointeur qu'un registre dont on sait, **pour cette fonction précise**, qu'il porte une chaîne. Lire `rdx` d'un `memmem` (une longueur) comme un pointeur produit n'importe quoi.

Étape 3 - Lire les arguments des appels



Question

Quelle chaîne est passée à chaque appel “intéressant” ? Pouvez-vous nommer chaque vérification uniquement à partir de ces chaînes ?

Ce que cette approche ne voit pas

- Certaines vérifications évitent volontairement la libc : **self-checksum** (relit son propre code), scan de prologue après un `dlsym`, vérification d'adresse de retour ou de GOT
- On voit leur **préparation** (le `dlsym`, la lecture de `/proc/self/maps`) mais pas leur **décision**



Question

Comment intercepter ces vérifications-là ? À quel niveau d'instrumentation faut-il redescendre, et quel évènement observer à la place de `EXEC_TRANSFER_CALL` ?



Bonus - Détecter une relecture de code

- Indice : une fonction qui relit **son propre segment exécutable** est suspecte, le code normal n'a jamais besoin de se lire lui-même
- `pyqbd1.getCurrentProcessMaps(True)` + `pyqbd1.PF_EXEC` pour délimiter le segment de code de la bibliothèque
- `vm.addMemRangeCB` sur cette plage avec `pyqbd1.MEMORY_READ`

```
1 vm.addMemRangeCB(CODE_LO, CODE_HI, pyqbd1.MEMORY_READ, cb, None)
```

 Python

Questions

Quelle plage de code est relue, et par quelle fonction ? Combien d'instructions distinctes font la lecture, et qu'est-ce qu'un balayage octet par octet, sans trou, vous apprend ?

Observer une primitive cryptographique

Une bibliothèque strippée, sans aucun symbole utile

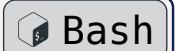


Cible

`libAES.so`, bibliothèque strippée, un seul export

- La table des symboles dynamiques **doit** rester intacte : c'est elle qui permet à d'autres programmes de trouver les fonctions exportées
- `strip` ne peut pas la supprimer sans casser le chargement dynamique

```
1 nm -D libAES.so
```



- `T` = défini (notre cible), `U` = importé (libc/libstdc++), `W` = template faible



Retrouver la convention d'appel sans header

- Désassembler directement à l'adresse trouvée

```
1 objdump -d -M intel --start-address=0xADDR \  
2 --stop-address=0xADDR+0x200 libAES.so
```



- Compter les premiers `mov [rbp-N], rdi/rsi/rdx/rcx/r8` : le prologue recopie chaque argument reçu vers la pile, autant de `mov` que d'arguments
- Repérer les appels `memcpy@plt` : leurs tailles d'opérandes indiquent quel argument est un buffer de données, lequel est une clé



Question

Combien d'arguments la fonction prend-elle ? Quel calcul de taille, visible dans le désassemblage, trahit un chiffrement par blocs ?



Appeler la fonction inconnue

- Une fois la convention d'appel reconstruite, `ctypes` n'a besoin que du nom du symbole, pas d'un header

```
1 lib.unknown_fn.restype = ctypes.POINTER(ctypes.c_ubyte)
2 result = lib.unknown_fn(data, len(data), key, len(key),
3                           ctypes.byref(out_len))
```



Classer chaque accès mémoire

- Sous QBDI, `vm.addMemAccessCB` déclenche un callback sur **tout** accès mémoire de l'instruction courante, peu importe l'adresse

```
1 vm.addMemAccessCB(pyqbdι.MEMORY_READ_WRITE, cb, data)
```

 Python

- Dans le callback, classer chaque accès selon la zone qu'il touche (pile virtuelle, buffer d'entrée, clé, ailleurs) à partir de `acc.accessAddress`



Question

En traçant l'ordre des accès contre leur adresse, quel motif domine : une bande plate à quelques adresses, ou un balayage diagonal sur de nombreuses adresses ? Que signifierait chacun pour un chiffrement par blocs ?

Le motif qui émerge

- Une **bande plate** `stack_*` domine : la matrice d'état 4x4 réécrite en place à chaque round, plus un petit cluster `other_*` (S-box et T-tables en `.rodata`)
- Un **balayage diagonal** signifierait au contraire une simple copie ou un remplissage de buffer, fait une seule fois
- `data_ptr` et `key_ptr` ne sont lus que **16 fois chacun**, puis plus jamais : la cible les copie sur la pile et chiffre la copie



À retenir

Les buffers d'entrée sont silencieux par **flot de données** : lus une fois, copiés, abandonnés. Le vrai travail est la fonction de round sur son état interne, pas sur votre texte clair.

Bonus - Compter les rounds AES depuis la trace

- La bande `stack_*` est la fonction de round. On peut **compter les rounds** en isolant une opération signature
- `SubBytes` fait 16 lookups S-box par round : bucketiser les lectures non-stack par adresse d'instruction (`acc.instAddress`) pour isoler ce PC

$$\text{lookups SubBytes} = \text{blocs} \times \text{Nr} \times 16$$



Questions

- Pour 16 octets en entrée : combien de blocs après padding, combien de rounds ?
- Comment recouper avec `MixColumns`, sautée au dernier round ?

Récapitulatif des APIs vues

Thank you

Contact information

Email:

vhoual@quarkslab.com

Phone:

+33 1 58 30 81 51

Website:

<https://www.quarkslab.com>